铁岭市医保影像云统一服务门户

生产环境部署与运维手册

文档版本：v1.0.0
适用系统：Tieling Healthcare Security Imaging Cloud Portal
技术栈：Spring Boot 2.3.12 + Vue CLI 3 + SQLite + Redis + Nginx
编制日期：2025年
密级：内部使用

目录

1. 部署架构概述
2. 服务器环境要求
3. 基础环境安装
4. 后端服务部署
5. 前端静态资源部署
6. Nginx 反向代理配置
7. Redis 生产配置
8. SQLite 数据库管理
9. 安全加固
10. 性能优化
11. 日志管理
12. 监控与告警
13. 备份与恢复
14. 常见问题排查

15. 版本升级流程

---

1. 部署架构概述

1.1 推荐生产架构

本系统推荐采用单机部署（适合铁岭市级政务系统规模），通过 Nginx 作为统一入口，实现前后端分离部署、SSL 终止和反向代理。

```
                    ┌──────────────────────────────────────────┐
                    │                 生产服务器                 │
                    │                                          │
用户浏览器  ──HTTPS──▶  Nginx (443/80)                        │
                    │       │                                  │
                    │       ├──▶ /          静态文件 (Vue)      │
                    │       │    /assets/                      │
                    │       │                                  │
                    │       └──▶ /api/      Spring Boot        │
                    │                 (8080)                   │
                    │                   │                      │
                    │                   ├──▶ Redis (6379)      │
                    │                   └──▶ SQLite (文件)     │
                    └──────────────────────────────────────────┘
```

1.2 端口规划

服务	端口	说明
Nginx HTTPS	443	对外服务端口（主）
Nginx HTTP	80	强制跳转至 HTTPS
Spring Boot	8080	后端服务（仅内网）
Redis	6379	缓存服务（仅内网）

1.3 目录规划

目录	用途
`/opt/yibao-portal/`	应用根目录
`/opt/yibao-portal/backend/`	后端 jar 包及配置
`/opt/yibao-portal/frontend/`	前端静态文件
`/opt/yibao-portal/data/`	SQLite 数据库文件
`/opt/yibao-portal/logs/`	应用日志
`/opt/yibao-portal/backup/`	数据备份
`/etc/nginx/conf.d/`	Nginx 配置

---

2. 服务器环境要求

2.1 最低配置（测试/小规模生产）

资源	最低要求	推荐配置
CPU	2 核	4 核
内存	4 GB	8 GB
磁盘	50 GB SSD	100 GB SSD
操作系统	CentOS 7.9 / Ubuntu 20.04 LTS	Ubuntu 22.04 LTS
带宽	5 Mbps	10 Mbps

> ***说明***：*SQLite 适合并发量较低的政务场景（日活 < 500 用户）。若未来接入全市所有医疗机构并发访问量显著增长，建议迁移至 MySQL 8.0 或 PostgreSQL 14。*

2.2 操作系统要求

推荐使用 **Ubuntu 22.04 LTS**，原因如下：长期支持至 2027 年，软件包版本较新，社区支持完善。生产环境应关闭不必要的服务，最小化安装。

---

3. 基础环境安装

3.1 安装 JDK 11

虽然代码兼容 Java 8，生产环境推荐使用 JDK 11 LTS，其安全补丁更新至 2026 年。

```
# Ubuntu 22.04
sudo apt update
sudo apt install -y openjdk-11-jdk

# 验证安装
java -version
# 预期输出：openjdk version "11.0.x"

# 设置 JAVA_HOME
echo 'export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64' >>
/etc/profile.d/java.sh
echo 'export PATH=$PATH:$JAVA_HOME/bin' >> /etc/profile.d/java.sh
source /etc/profile.d/java.sh
```

3.2 安装 Redis 6

```
sudo apt install -y redis-server

# 验证安装
redis-cli ping
# 预期输出：PONG
```

3.3 安装 Nginx

```
sudo apt install -y nginx

# 验证安装
nginx -v
# 预期输出：nginx version: nginx/1.18.x 或更高

# 设置开机自启
sudo systemctl enable nginx
sudo systemctl start nginx
```

3.4 安装 Node.js（仅构建前端使用）

```
# 使用 NodeSource 安装 Node.js 18 LTS
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs

node -v   # v18.x.x
npm -v    # 9.x.x
```

3.5 创建应用目录和专用用户

```
# 创建专用运行用户（不允许登录）
sudo useradd -r -s /sbin/nologin yibao

# 创建目录结构
sudo mkdir -p /opt/yibao-portal/{backend,frontend,data,logs,backup}

# 设置目录权限
sudo chown -R yibao:yibao /opt/yibao-portal/
sudo chmod 750 /opt/yibao-portal/data/
sudo chmod 750 /opt/yibao-portal/logs/
```

---

4. 后端服务部署

4.1 构建生产 jar 包

在开发机或 CI 环境中执行：

```
cd backend

# 生产环境构建（跳过测试）
mvn clean package -DskipTests -Pprod

# 构建产物
ls -lh target/imaging-cloud-portal-1.0.0.jar
```

4.2 上传并部署 jar 包

```
# 上传至服务器
scp target/imaging-cloud-portal-1.0.0.jar \
    user@your-server:/opt/yibao-portal/backend/

# 创建生产配置文件（不要将密码写入 jar 包内）
sudo nano /opt/yibao-portal/backend/application-prod.yml
```

4.3 生产环境配置文件

创建 `/opt/yibao-portal/backend/application-prod.yml`，**覆盖**默认配置中的敏感项：

```yaml
server:
  port: 8080
  # 生产环境建议启用压缩
  compression:
    enabled: true
    mime-types:
application/json,application/xml,text/html,text/xml,text/plain
    min-response-size: 1024

spring:
  datasource:
    # 数据库文件存放在专用目录，便于备份
    url: jdbc:sqlite:/opt/yibao-portal/data/imaging_cloud.db
    driver-class-name: org.sqlite.JDBC

  jpa:
    hibernate:
      # 生产环境使用 validate，避免意外修改表结构
      ddl-auto: validate
    show-sql: false

  redis:
    host: 127.0.0.1
    port: 6379
    # 生产环境必须设置密码
    password: ${REDIS_PASSWORD}
    database: 0
    timeout: 3000ms
    lettuce:
      pool:
        max-active: 20
        max-idle: 10
        min-idle: 5
        max-wait: 2000ms

# JWT 配置（生产环境必须替换密钥）
jwt:
  # 使用环境变量注入，避免密钥硬编码
  secret: ${JWT_SECRET}
  expiration: 7200000       # 2小时
  refresh-expiration: 604800000  # 7天
  header: Authorization
  prefix: Bearer

# SSO 子系统配置（按实际部署地址修改）
```

```yaml
sso:
  whitelist:
    - /auth/login
    - /auth/logout
    - /auth/refresh
    - /public/**
    - /actuator/health
  subsystems:
    supervisor-url: https://your-domain.com/supervisor/
    hospital-url: https://your-domain.com/hospital/
    patient-url: https://your-domain.com/patient/
    developer-url: https://your-domain.com/developer/

# 日志配置
logging:
  level:
    root: WARN
    cn.tieling.yibao: INFO
    org.springframework.web: WARN
  file:
    name: /opt/yibao-portal/logs/imaging-cloud.log
    max-size: 100MB
    max-history: 30
  pattern:
    file: "%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{50} -
%msg%n"

# Actuator 健康检查（仅开放 health 端点）
management:
  endpoints:
    web:
      exposure:
        include: health,info
  endpoint:
    health:
      show-details: when-authorized
```

4.4 配置环境变量

强烈建议通过环境变量注入敏感配置，避免密码写入配置文件：

```
# 创建环境变量文件
sudo nano /opt/yibao-portal/backend/.env

# 写入以下内容（替换为实际值）
REDIS_PASSWORD=your_strong_redis_password_here
JWT_SECRET=your_64_char_random_secret_key_here_please_change_this

# 限制文件权限（仅 yibao 用户可读）
sudo chown yibao:yibao /opt/yibao-portal/backend/.env
sudo chmod 600 /opt/yibao-portal/backend/.env
```

4.5 配置 Systemd 服务

创建 `/etc/systemd/system/yibao-portal.service`：

```
[Unit]
Description=铁岭市医保影像云统一服务门户 - 后端服务
Documentation=https://github.com/douyuan163-git/Yinaoyingxiangyun
After=network.target redis.service
Wants=redis.service

[Service]
Type=simple
User=yibao
Group=yibao
WorkingDirectory=/opt/yibao-portal/backend

# 从环境变量文件加载敏感配置
EnvironmentFile=/opt/yibao-portal/backend/.env

# 启动命令：指定生产配置文件
ExecStart=/usr/bin/java \
    -Xms512m \
    -Xmx1024m \
    -XX:+UseG1GC \
    -XX:MaxGCPauseMillis=200 \
    -XX:+HeapDumpOnOutOfMemoryError \
    -XX:HeapDumpPath=/opt/yibao-portal/logs/heapdump.hprof \
    -Djava.security.egd=file:/dev/./urandom \
    -Dspring.profiles.active=prod \
    -Dspring.config.additional-location=/opt/yibao-
portal/backend/application-prod.yml \
    -jar /opt/yibao-portal/backend/imaging-cloud-portal-1.0.0.jar

# 优雅停机等待时间
TimeoutStopSec=30
KillSignal=SIGTERM
SendSIGKILL=no

# 自动重启策略
Restart=on-failure
RestartSec=10
StartLimitInterval=60
StartLimitBurst=3

# 安全限制
NoNewPrivileges=true
PrivateTmp=true

# 标准输出重定向
```

```
StandardOutput=append:/opt/yibao-portal/logs/stdout.log
StandardError=append:/opt/yibao-portal/logs/stderr.log

[Install]
WantedBy=multi-user.target
```

```
# 重载 systemd 配置
sudo systemctl daemon-reload

# 启动服务
sudo systemctl start yibao-portal

# 设置开机自启
sudo systemctl enable yibao-portal

# 查看服务状态
sudo systemctl status yibao-portal

# 查看实时日志
sudo journalctl -u yibao-portal -f
```

4.6 验证后端服务

```
# 健康检查
curl http://localhost:8080/api/actuator/health

# 预期响应
# {"status":"UP"}

# 测试登录接口
curl -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json; charset=utf-8" \
  -d '{"username":"admin","password":"admin123"}'
```

---

5. 前端静态资源部署

5.1 构建生产版本

在开发机执行：

```
cd frontend

# 安装依赖
npm ci  # 使用 ci 命令确保依赖版本锁定

# 生产构建
npm run build

# 构建产物在 dist/ 目录
ls -lh dist/
```

5.2 上传静态文件

```
# 上传至服务器
rsync -avz --delete dist/ \
    user@your-server:/opt/yibao-portal/frontend/

# 设置文件权限
sudo chown -R nginx:nginx /opt/yibao-portal/frontend/
sudo find /opt/yibao-portal/frontend/ -type f -exec chmod 644 {} \;
sudo find /opt/yibao-portal/frontend/ -type d -exec chmod 755 {} \;
```

---

6. Nginx 反向代理配置

6.1 申请 SSL 证书

推荐使用 Let’s Encrypt 免费证书（政务域名也可向 CA 机构申请 OV 证书）：

```
# 安装 certbot
sudo apt install -y certbot python3-certbot-nginx

# 申请证书（替换为实际域名）
sudo certbot --nginx -d your-domain.com

# 证书自动续期（certbot 已自动配置定时任务）
sudo certbot renew --dry-run
```

6.2 Nginx 主配置

创建 `/etc/nginx/conf.d/yibao-portal.conf`：

```
# 铁岭市医保影像云统一服务门户 - Nginx 配置
# 版本：v1.0.0

# HTTP → HTTPS 强制跳转
server {
    listen 80;
    server_name your-domain.com;

    # 健康检查路径（负载均衡器探活使用）
    location /health {
        return 200 'OK';
        add_header Content-Type text/plain;
    }

    # 其他请求强制跳转 HTTPS
    location / {
        return 301 https://$server_name$request_uri;
    }
}

# HTTPS 主配置
server {
    listen 443 ssl http2;
    server_name your-domain.com;

    # ==================== SSL 配置 ====================
    ssl_certificate     /etc/letsencrypt/live/your-domain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/your-domain.com/privkey.pem;

    # 仅允许 TLS 1.2 和 1.3（禁用不安全的旧版本）
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-RSA-AES128-GCM-SHA256:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-
RSA-CHACHA20-POLY1305;
    ssl_prefer_server_ciphers off;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 1d;
    ssl_session_tickets off;

    # HSTS（强制浏览器使用 HTTPS，有效期1年）
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;

    # ==================== 安全响应头 ====================
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
```

```
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com; img-src 'self' data: https://d2xsxph8kpxj0f.cloudfront.net; connect-src 'self'" always;

    # ==================== 访问日志 ====================
    access_log /opt/yibao-portal/logs/nginx-access.log combined buffer=16k flush=5s;
    error_log  /opt/yibao-portal/logs/nginx-error.log warn;

    # ==================== 前端静态文件 ====================
    root /opt/yibao-portal/frontend;
    index index.html;

    # Vue Router history 模式支持
    location / {
        try_files $uri $uri/ /index.html;

        # HTML 文件不缓存（确保更新及时生效）
        location ~* \.html$ {
            add_header Cache-Control "no-cache, no-store, must-revalidate";
            add_header Pragma "no-cache";
        }
    }

    # 静态资源长期缓存（Vue CLI 构建产物带 hash 指纹）
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
        access_log off;
    }

    # ==================== 后端 API 反向代理 ====================
    location /api/ {
        proxy_pass http://127.0.0.1:8080/api/;

        # 代理头设置
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # 超时配置
        proxy_connect_timeout 10s;
```

```
        proxy_send_timeout 30s;
        proxy_read_timeout 30s;

        # 禁用缓冲（适合 JSON API）
        proxy_buffering off;

        # 上传文件大小限制
        client_max_body_size 10m;
    }

    # ==================== 安全限制 ====================
    # 禁止访问隐藏文件（.git, .env 等）
    location ~ /\. {
        deny all;
        access_log off;
        log_not_found off;
    }

    # 禁止访问备份文件
    location ~* \.(bak|sql|tar|gz|zip)$ {
        deny all;
    }

    # ==================== 限流配置 ====================
    # 登录接口限流（防暴力破解）
    location /api/auth/login {
        limit_req zone=login_limit burst=5 nodelay;
        proxy_pass http://127.0.0.1:8080/api/auth/login;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

在 `/etc/nginx/nginx.conf` 的 `http {}` 块中添加限流区定义：

```
 # 登录接口限流：每个 IP 每分钟最多 10 次请求
limit_req_zone $binary_remote_addr zone=login_limit:10m rate=10r/m;

# API 接口通用限流：每个 IP 每秒最多 30 次请求
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=30r/s;
```

```
# 测试 Nginx 配置语法
sudo nginx -t

# 重载配置（不中断服务）
sudo nginx -s reload
```

---

7. Redis 生产配置

7.1 修改 Redis 配置文件

编辑 `/etc/redis/redis.conf`：

```
sudo nano /etc/redis/redis.conf
```

关键配置项（修改以下内容）：

```
# 绑定本机地址，禁止外网访问
bind 127.0.0.1

# 设置强密码（与 application-prod.yml 中的 REDIS_PASSWORD 一致）
requirepass your_strong_redis_password_here

# 禁用危险命令（防止误操作清空数据）
rename-command FLUSHALL ""
rename-command FLUSHDB ""
rename-command CONFIG "CONFIG_DISABLED_IN_PROD"
rename-command DEBUG ""

# 最大内存限制（根据服务器内存调整，建议设为总内存的 25%）
maxmemory 512mb

# 内存淘汰策略：优先淘汰最近最少使用的 key
maxmemory-policy allkeys-lru

# 持久化配置（AOF 模式，保证数据不丢失）
appendonly yes
appendfsync everysec
no-appendfsync-on-rewrite no

# 日志级别
loglevel notice
logfile /var/log/redis/redis-server.log

# 慢查询日志（超过 10ms 的命令记录）
slowlog-log-slower-than 10000
slowlog-max-len 128
```

```
# 重启 Redis 使配置生效
sudo systemctl restart redis

# 验证密码认证
redis-cli -a your_strong_redis_password_here ping
# 预期输出：PONG
```

7.2 Redis 连接验证

```
# 验证 Token 存储功能
redis-cli -a your_password
> AUTH your_password
> KEYS yibao:*
> INFO memory
```

---

8. SQLite 数据库管理

8.1 数据库初始化

首次部署时，Spring Boot 会通过 `DataInitializer` 自动创建表结构并写入演示数据。生产环境中，建议在首次启动后**立即修改演示账号密码**。

```
# 安装 SQLite 命令行工具
sudo apt install -y sqlite3

# 连接数据库
sqlite3 /opt/yibao-portal/data/imaging_cloud.db

# 查看表结构
.tables
.schema sys_user

# 修改管理员密码（密码需与后端加密方式一致）
# 注意：当前版本使用明文密码比较，生产环境强烈建议升级为 BCrypt
UPDATE sys_user SET password = 'new_strong_password' WHERE username =
'admin';

.quit
```

> ***重要安全提示：***当前后端代码使用明文密码比较（`password.equals(user.getPassword())`），这仅适合演示环境。生产部署前，**必须**将密码改为 BCrypt 哈希存储，参见第 9.3 节。

8.2 数据库文件权限

```
# 确保数据库文件仅 yibao 用户可访问
sudo chown yibao:yibao /opt/yibao-portal/data/imaging_cloud.db
sudo chmod 640 /opt/yibao-portal/data/imaging_cloud.db
```

8.3 SQLite 性能优化

在应用启动时，通过 JPA 执行以下 PRAGMA 优化 SQLite 性能：

在 `DataInitializer.java` 中添加：

```
@PostConstruct
public void optimizeSQLite() {
    entityManager.createNativeQuery("PRAGMA
journal_mode=WAL").executeUpdate();
    entityManager.createNativeQuery("PRAGMA
synchronous=NORMAL").executeUpdate();
    entityManager.createNativeQuery("PRAGMA
cache_size=10000").executeUpdate();
    entityManager.createNativeQuery("PRAGMA
temp_store=MEMORY").executeUpdate();
}
```

> ***WAL 模式说明***：*Write-Ahead Logging 模式允许读写并发，显著提升多用户场景下的读取性能，是 SQLite 生产环境的推荐配置。*

---

9. 安全加固

9.1 操作系统安全

```
# 1. 禁用 root 远程登录
sudo sed -i 's/PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config
sudo systemctl restart sshd

# 2. 配置防火墙（仅开放必要端口）
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp    # SSH（建议改为非标准端口）
sudo ufw allow 80/tcp    # HTTP（跳转用）
sudo ufw allow 443/tcp   # HTTPS
sudo ufw enable

# 3. 禁止 8080 端口外网访问（仅 Nginx 内部访问）
sudo ufw deny 8080/tcp
sudo ufw deny 6379/tcp

# 4. 安装 fail2ban 防暴力破解
sudo apt install -y fail2ban
sudo systemctl enable fail2ban
sudo systemctl start fail2ban
```

9.2 JWT 密钥安全规范

生产环境 JWT 密钥必须满足以下要求：

要求	说明
长度	至少 64 个字符（512 位）
字符集	大小写字母 + 数字 + 特殊字符
存储方式	环境变量，不得写入代码或配置文件
轮换周期	建议每 6 个月轮换一次
备份	加密存储，与数据库备份分开保管

生成强密钥的方法：

```
# 生成 64 字符随机密钥
openssl rand -base64 48 | tr -d '\n'
```

9.3 密码存储升级（必须执行）

当前版本使用明文密码，**生产部署前必须升级为 BCrypt**。

在 `pom.xml` 中已包含 `spring-security-crypto`，在 `AuthServiceImpl.java` 中修改：

```
// 在类中注入 PasswordEncoder
@Autowired
private BCryptPasswordEncoder passwordEncoder;

// 登录验证（替换原有明文比较）
if (!passwordEncoder.matches(request.getPassword(), user.getPassword())) {
    throw new RuntimeException("密码错误");
}
```

在 `SecurityConfig.java`（新建）中注册 Bean：

```
@Bean
public BCryptPasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12);  // 强度系数12，平衡安全性与性能
}
```

数据库中现有密码需要重新加密：

```
# 使用 Spring Boot 临时工具类生成 BCrypt 哈希
# 或使用以下 Python 脚本
python3 -c "import bcrypt; print(bcrypt.hashpw(b'admin123',
bcrypt.gensalt(12)).decode())"
```

9.4 敏感数据脱敏

接口返回的用户信息中，身份证号、手机号等敏感字段必须脱敏处理：

```
// 示例：手机号脱敏
public static String maskPhone(String phone) {
    if (phone == null || phone.length() < 7) return phone;
    return phone.substring(0, 3) + "****" + phone.substring(7);
}

// 示例：身份证号脱敏
public static String maskIdCard(String idCard) {
    if (idCard == null || idCard.length() < 10) return idCard;
    return idCard.substring(0, 4) + "**********" + idCard.substring(14);
}
```

9.5 SQL 注入防护

本项目使用 Spring Data JPA，所有查询通过参数化方式执行，已具备基本的 SQL 注入防护。但需注意：

- **禁止**在 `@Query` 注解中使用字符串拼接
- 所有用户输入必须通过 `@Valid` 注解进行参数校验
- 日志中禁止打印完整的 SQL 语句（`show-sql: false`）

---

10. 性能优化

10.1 JVM 参数调优

根据服务器内存调整 JVM 参数（在 systemd 服务文件中修改）：

服务器内存	推荐 JVM 参数
4 GB	`-Xms512m -Xmx1024m`
8 GB	`-Xms1g -Xmx2g`
16 GB	`-Xms2g -Xmx4g`

```
# 完整推荐 JVM 参数（4GB 服务器）
-Xms512m
-Xmx1024m
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200
-XX:G1HeapRegionSize=16m
-XX:+UseStringDeduplication
-XX:+OptimizeStringConcat
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/opt/yibao-portal/logs/
-Djava.security.egd=file:/dev/./urandom
```

10.2 Spring Boot 连接池优化

在 `application-prod.yml` 中添加 HikariCP 配置（Spring Boot 默认连接池）：

```yaml
spring:
  datasource:
    hikari:
      # SQLite 单文件数据库，连接池不宜过大
      maximum-pool-size: 5
      minimum-idle: 2
      connection-timeout: 20000
      idle-timeout: 300000
      max-lifetime: 1200000
      # SQLite WAL 模式下的连接初始化
      connection-init-sql: "PRAGMA journal_mode=WAL; PRAGMA
synchronous=NORMAL;"
```

10.3 Redis 缓存策略

对高频访问的静态数据（政策列表、机构列表）增加 Redis 缓存，减少数据库压力：

```
// 在 PolicyController 中添加缓存注解
@Cacheable(value = "policy-list", key = "#level + '_' + #page")
public Result<Page<Policy>> list(@RequestParam(required = false) String
level,
                                 @RequestParam(defaultValue = "0") int
page) {
    // ...
}

// 在 application-prod.yml 中配置缓存
spring:
  cache:
    type: redis
    redis:
      time-to-live: 300000  # 5分钟过期
```

10.4 Nginx 性能优化

在 `/etc/nginx/nginx.conf` 中调整：

```
 worker_processes auto;  # 自动匹配 CPU 核数
worker_rlimit_nofile 65535;

events {
    worker_connections 4096;
    use epoll;
    multi_accept on;
}

http {
    # 开启 sendfile 零拷贝
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;

    # 连接超时
    keepalive_timeout 65;
    keepalive_requests 1000;

    # Gzip 压缩
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_comp_level 6;
    gzip_types text/plain text/css application/json application/javascript
               text/xml application/xml application/xml+rss text/javascript
               application/vnd.ms-fontobject application/x-font-ttf
               font/opentype image/svg+xml image/x-icon;

    # 隐藏版本号
    server_tokens off;

    # 客户端请求体大小限制
    client_max_body_size 10m;
    client_body_buffer_size 128k;
}
```

---

11. 日志管理

11.1 日志文件说明

日志文件	说明	保留策略
`/opt/yibao-portal/logs/imaging-cloud.log`	应用主日志	按天滚动，保留 30 天
`/opt/yibao-portal/logs/stdout.log`	标准输出	保留 7 天
`/opt/yibao-portal/logs/stderr.log`	错误输出	保留 30 天
`/opt/yibao-portal/logs/nginx-access.log`	Nginx 访问日志	按天滚动，保留 90 天
`/opt/yibao-portal/logs/nginx-error.log`	Nginx 错误日志	保留 30 天
`/var/log/redis/redis-server.log`	Redis 日志	保留 30 天

11.2 配置 logrotate

创建 `/etc/logrotate.d/yibao-portal`：

```
/opt/yibao-portal/logs/*.log {
    daily
    rotate 30
    compress
    delaycompress
    missingok
    notifempty
    sharedscripts
    postrotate
        # Nginx 日志切割后重新打开文件
        /bin/kill -USR1 $(cat /run/nginx.pid 2>/dev/null) 2>/dev/null ||
true
    endscript
}
```

11.3 关键日志监控

以下日志模式需要重点关注：

```
# 监控登录失败（可能的暴力破解）
grep "登录失败\|密码错误\|用户不存在" /opt/yibao-portal/logs/imaging-cloud.log |
tail -50

# 监控 JWT 验证失败（可能的 Token 伪造）
grep "Token验证失败\|Token已失效\|401" /opt/yibao-portal/logs/imaging-
cloud.log | tail -50

# 监控 Nginx 4xx/5xx 错误
awk '$9 >= 400' /opt/yibao-portal/logs/nginx-access.log | tail -50
```

---

12. 监控与告警

12.1 服务健康检查脚本

创建 `/opt/yibao-portal/scripts/health-check.sh`：

```bash
#!/bin/bash
# 铁岭市医保影像云门户 - 健康检查脚本

LOG_FILE="/opt/yibao-portal/logs/health-check.log"
ALERT_EMAIL="admin@tieling-yibao.gov.cn"
TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')

check_service() {
    local name=$1
    local check_cmd=$2

    if eval "$check_cmd" > /dev/null 2>&1; then
        echo "[$TIMESTAMP] ✓ $name 正常" >> $LOG_FILE
    else
        echo "[$TIMESTAMP] ✗ $name 异常！" >> $LOG_FILE
        # 发送告警邮件（需配置 mailutils）
        echo "$name 服务异常，请立即检查！时间：$TIMESTAMP" | \
            mail -s "【告警】铁岭医保影像云 $name 异常" $ALERT_EMAIL
    fi
}

# 检查后端服务
check_service "Spring Boot" "curl -sf
http://localhost:8080/api/actuator/health"

# 检查 Redis
check_service "Redis" "redis-cli -a $REDIS_PASSWORD ping | grep -q PONG"

# 检查 Nginx
check_service "Nginx" "nginx -t"

# 检查磁盘空间（超过 85% 告警）
DISK_USAGE=$(df /opt/yibao-portal/data | awk 'NR==2 {print $5}' | tr -d '%')
if [ "$DISK_USAGE" -gt 85 ]; then
    echo "[$TIMESTAMP] ✗ 磁盘使用率 ${DISK_USAGE}%，请及时清理！" >> $LOG_FILE
fi
```

```
# 设置执行权限
chmod +x /opt/yibao-portal/scripts/health-check.sh

# 配置定时任务（每5分钟检查一次）
echo "*/5 * * * * yibao /opt/yibao-portal/scripts/health-check.sh" | \
    sudo tee -a /etc/crontab
```

12.2 关键监控指标

指标	告警阈值	说明
CPU 使用率	> 80% 持续 5 分钟	可能存在性能问题
内存使用率	> 85%	考虑增加内存或优化
磁盘使用率	> 85%	清理日志或扩容
Spring Boot 响应时间	> 2000ms	API 性能下降
Redis 内存使用	> maxmemory 的 80%	调整缓存策略
登录失败次数	> 10次/分钟/IP	可能的暴力破解
HTTP 5xx 错误率	> 1%	后端服务异常

---

13. 备份与恢复

13.1 自动备份脚本

创建 `/opt/yibao-portal/scripts/backup.sh`：

```bash
#!/bin/bash
# 铁岭市医保影像云门户 - 数据备份脚本

BACKUP_DIR="/opt/yibao-portal/backup"
DATA_DIR="/opt/yibao-portal/data"
DATE=$(date '+%Y%m%d_%H%M%S')
BACKUP_FILE="$BACKUP_DIR/imaging_cloud_$DATE.db"
KEEP_DAYS=30  # 保留最近30天的备份

# 创建备份目录
mkdir -p $BACKUP_DIR

# SQLite 在线备份（使用 .backup 命令，支持热备份）
sqlite3 $DATA_DIR/imaging_cloud.db ".backup $BACKUP_FILE"

# 压缩备份文件
gzip $BACKUP_FILE

echo "[$(date '+%Y-%m-%d %H:%M:%S')] 备份完成：${BACKUP_FILE}.gz"

# 删除超过保留期的备份
find $BACKUP_DIR -name "imaging_cloud_*.db.gz" -mtime +$KEEP_DAYS -delete

echo "[$(date '+%Y-%m-%d %H:%M:%S')] 清理完成，保留最近 $KEEP_DAYS 天备份"
```

```bash
chmod +x /opt/yibao-portal/scripts/backup.sh

# 配置每日凌晨2点自动备份
echo "0 2 * * * yibao /opt/yibao-portal/scripts/backup.sh >> /opt/yibao-portal/logs/backup.log 2>&1" | \
    sudo tee -a /etc/crontab
```

13.2 数据恢复流程

```
# 1. 停止后端服务
sudo systemctl stop yibao-portal

# 2. 备份当前数据库（以防恢复失败）
cp /opt/yibao-portal/data/imaging_cloud.db \
   /opt/yibao-portal/data/imaging_cloud.db.before-restore

# 3. 解压备份文件
gunzip /opt/yibao-portal/backup/imaging_cloud_20250101_020000.db.gz

# 4. 恢复数据库
cp /opt/yibao-portal/backup/imaging_cloud_20250101_020000.db \
   /opt/yibao-portal/data/imaging_cloud.db

# 5. 验证数据库完整性
sqlite3 /opt/yibao-portal/data/imaging_cloud.db "PRAGMA integrity_check;"
# 预期输出：ok

# 6. 重启服务
sudo systemctl start yibao-portal

# 7. 验证服务正常
curl http://localhost:8080/api/actuator/health
```

---

14. 常见问题排查

14.1 后端服务无法启动

症状： `systemctl status yibao-portal` 显示 `failed`

```
# 查看详细错误信息
sudo journalctl -u yibao-portal -n 100 --no-pager

# 常见原因及解决方案：

# 原因1：Redis 连接失败
# 错误：Unable to connect to Redis; nested exception is
io.lettuce.core.RedisConnectionException
# 解决：检查 Redis 是否运行，密码是否正确
sudo systemctl status redis
redis-cli -a your_password ping

# 原因2：端口被占用
# 错误：Address already in use: bind
# 解决：查找并终止占用 8080 端口的进程
sudo lsof -i :8080
sudo kill -9 <PID>

# 原因3：数据库文件权限不足
# 错误：unable to open database file
# 解决：检查数据库文件权限
ls -la /opt/yibao-portal/data/
sudo chown yibao:yibao /opt/yibao-portal/data/imaging_cloud.db
```

14.2 前端页面无法访问

```
# 检查 Nginx 状态
sudo systemctl status nginx
sudo nginx -t

# 检查静态文件是否存在
ls -la /opt/yibao-portal/frontend/index.html

# 检查 Nginx 错误日志
tail -50 /opt/yibao-portal/logs/nginx-error.log

# 检查 SSL 证书是否过期
openssl x509 -in /etc/letsencrypt/live/your-domain.com/cert.pem -noout -
dates
```

14.3 API 请求返回 401

```
# 检查 JWT Token 是否过期
# 解码 Token（Base64 解码 payload 部分）
echo "your_token_payload" | base64 -d | python3 -m json.tool

# 检查 Redis 中 Token 是否被加入黑名单
redis-cli -a your_password KEYS "yibao:blacklist:*"

# 检查系统时间是否同步（JWT 依赖时间戳）
timedatectl status
sudo timedatectl set-ntp true
```

14.4 数据库锁定错误

```
# SQLite 出现 "database is locked" 错误
# 原因：多个进程同时写入，或上次异常退出留下锁文件

# 检查是否有残留锁文件
ls -la /opt/yibao-portal/data/imaging_cloud.db*

# 删除锁文件（确保应用已停止后操作）
sudo systemctl stop yibao-portal
rm -f /opt/yibao-portal/data/imaging_cloud.db-wal
rm -f /opt/yibao-portal/data/imaging_cloud.db-shm
sudo systemctl start yibao-portal
```

---

15. 版本升级流程

15.1 标准升级步骤

```
# 1. 通知用户（建议在业务低峰期操作，如凌晨2-4点）
# 在门户首页发布维护公告

# 2. 备份数据库
/opt/yibao-portal/scripts/backup.sh

# 3. 备份当前 jar 包
cp /opt/yibao-portal/backend/imaging-cloud-portal-1.0.0.jar \
   /opt/yibao-portal/backend/imaging-cloud-portal-1.0.0.jar.bak

# 4. 上传新版本
scp imaging-cloud-portal-1.0.1.jar \
    user@your-server:/opt/yibao-portal/backend/

# 5. 停止服务（等待优雅停机）
sudo systemctl stop yibao-portal
sleep 10

# 6. 替换 jar 包
mv /opt/yibao-portal/backend/imaging-cloud-portal-1.0.1.jar \
   /opt/yibao-portal/backend/imaging-cloud-portal-1.0.0.jar

# 7. 如有数据库结构变更，执行迁移脚本
# sqlite3 /opt/yibao-portal/data/imaging_cloud.db < migration_v1.0.1.sql

# 8. 启动新版本
sudo systemctl start yibao-portal

# 9. 验证服务正常
sleep 15
curl http://localhost:8080/api/actuator/health

# 10. 更新前端静态文件
rsync -avz --delete dist/ /opt/yibao-portal/frontend/
sudo nginx -s reload
```

15.2 回滚流程

若新版本出现严重问题，执行以下回滚：

```
# 1. 停止服务
sudo systemctl stop yibao-portal

# 2. 恢复旧版本 jar 包
cp /opt/yibao-portal/backend/imaging-cloud-portal-1.0.0.jar.bak \
   /opt/yibao-portal/backend/imaging-cloud-portal-1.0.0.jar

# 3. 恢复数据库（如有结构变更）
/opt/yibao-portal/scripts/restore.sh <backup_file>

# 4. 启动旧版本
sudo systemctl start yibao-portal

# 5. 验证
curl http://localhost:8080/api/actuator/health
```

附录 A：部署检查清单

在正式上线前，逐项确认以下检查项：

类别	检查项	状态
安全	JWT 密钥已替换为强随机密钥（64位+）	☐
安全	密码存储已升级为 BCrypt	☐
安全	Redis 已设置强密码	☐
安全	演示账号密码已修改	☐
安全	防火墙已配置，8080/6379 端口不对外开放	☐
安全	SSL 证书已配置，HTTP 强制跳转 HTTPS	☐
安全	安全响应头已配置（X-Frame-Options 等）	☐
性能	JVM 内存参数已根据服务器配置调整	☐
性能	Nginx Gzip 压缩已开启	☐
性能	静态资源缓存策略已配置	☐
可用性	Systemd 服务已配置开机自启	☐
可用性	健康检查脚本已配置	☐
可用性	自动备份脚本已配置（每日凌晨2点）	☐
运维	日志轮转已配置（logrotate）	☐
运维	磁盘空间监控已配置	☐
业务	子系统 SSO 跳转 URL 已更新为实际地址	☐
业务	数据初始化内容已替换为真实数据	☐

附录 B：技术支持联系方式

- **系统主办**：铁岭市医疗保障局信息中心
- **技术支持**：融御科技（辽宁）有限公司
- **GitHub 仓库**：https://github.com/douyuan163-git/Yinaoyingxiangyun
- **紧急联系**：如遇生产环境严重故障，请立即联系技术支持团队

本文档最后更新：2025年 | 版本 v1.0.0